

Web Application VAPT Assessment Report

OWASP Juice Shop - Local Lab Security Assessment

Web Application VAPT Assessment Report

Target: OWASP Juice Shop (`http://127.0.0.1:3000`) **Assessment Type:** Web Application Vulnerability Assessment and Penetration Testing (VAPT) **Environment:** Local, isolated lab (Kali Linux attacker machine, Juice Shop hosted via Docker)
Prepared as a personal portfolio/learning project

Executive Summary

This report documents a beginner-level web application security assessment conducted against a locally hosted instance of OWASP Juice Shop, an application intentionally built with known vulnerabilities for security training purposes. The assessment followed a structured methodology consisting of reconnaissance, enumeration, request analysis, authentication testing, and vulnerability validation.

One critical vulnerability was identified and confirmed: a SQL Injection flaw in the login endpoint (`POST /rest/user/login`) that allows an attacker to bypass authentication and gain access to an administrative account without valid credentials. This finding is documented in detail below, along with its impact and recommended remediation.

This assessment was limited in scope and depth, consistent with an entry-level learning exercise, and should not be considered a comprehensive penetration test.

1. Introduction

Web application security testing is a core skill in cybersecurity. This project was undertaken to gain hands-on experience with a structured VAPT methodology by testing OWASP Juice Shop, a deliberately vulnerable application designed for this purpose, in a private, isolated lab environment.

2. Scope

Testing was limited strictly to the local OWASP Juice Shop instance at `http://127.0.0.1:3000` . No other systems, networks, or third parties were involved. No testing was performed outside of this controlled local environment.

3. Testing Environment

Component	Detail
Attacking machine	Kali Linux
Target application	OWASP Juice Shop
Target hosting	Docker (local)
Network	Isolated local/virtual lab

4. Methodology

The assessment followed these stages:

1. Reconnaissance
2. Enumeration
3. Web application request analysis
4. Authentication testing
5. Vulnerability validation
6. Evidence collection
7. Impact assessment
8. Remediation recommendations

5. Tools Used

- **Nmap** — port/service scanning
- **Gobuster** — directory enumeration
- **Burp Suite** — HTTP traffic interception and manipulation
- **Kali Linux** — testing platform

- **Docker** — local application hosting

6. Reconnaissance

An Nmap scan was performed to confirm the target service:

```
nmap -sV -p 3000 127.0.0.1
```

Port 3000/tcp was found open. Service-version detection did not return a specific recognized label, but the HTTP response from the port included the page title `OWASP Juice Shop`, confirming the target application was active on this port. Output was saved to `01-reconnaissance/nmap-basic.txt` and `01-reconnaissance/nmap-service-version.txt`.

7. Enumeration

Directory enumeration was carried out using Gobuster with a standard common wordlist:

```
gobuster dir -u http://127.0.0.1:3000 -w /usr/share/wordlists/dirb/common.txt
```

Results were saved to `02-enumeration/gobuster.txt`.

8. Web Application Testing

Using Burp Suite, HTTP requests to the application were intercepted and analyzed. As an example, the following API request was replayed via Burp Repeater with a modified parameter:

```
GET /api/Challenges/?name=Score%20Board
```

modified to

```
GET /api/Challenges/?name=test
```

This returned `HTTP/1.1 200 OK` with an empty data set — normal application behavior, not indicative of a vulnerability.

9. Vulnerability Findings

#	Finding	Severity	Endpoint
1	SQL Injection leading to Authentication Bypass	Critical	<code>POST /rest/user/login</code>

10. Finding Details

Finding 01 — SQL Injection Leading to Authentication Bypass

Severity: Critical **Endpoint:** `POST /rest/user/login` **Parameter:** `email`

Baseline test — A normal login request with an invalid but well-formed email/password combination returned `HTTP/1.1 401 Unauthorized`, confirming standard authentication behavior.

Error-based test — Submitting a single quote (`'`) in the `email` field produced `HTTP/1.1 500 Internal Server Error`, exposing a SQLite database error and internal stack trace. This indicated unsafe handling of input in a SQL query, though it did not on its own confirm an authentication bypass.

Bypass validation — Submitting the payload `' OR 1=1--` in the `email` field returned `HTTP/1.1 200 OK`, with a valid authentication token issued for the account `admin@juice-sh.op`. This confirmed that authentication could be fully bypassed using SQL injection. The authentication token is represented as `[REDACTED]` throughout this report and was never published in full.

Full technical details are available in `04-findings/01-sql-injection-authentication-bypass.md`.

11. Impact Assessment

This vulnerability allows an unauthenticated attacker to bypass the login mechanism entirely and gain access to any account, including administrative accounts, without valid credentials. Depending on the privileges available to the compromised account,

this could lead to unauthorized access to sensitive data, modification of application content, or further compromise of the application. Given the critical nature of authentication bypass, this finding is rated **Critical**.

12. Remediation

- Use parameterized queries/prepared statements for all database interactions.
- Never concatenate user-controlled input directly into SQL statements.
- Apply strict server-side validation on all input fields, especially authentication-related fields.
- Return generic error messages to end users; do not expose internal error details or stack traces.
- Conduct a broader code review to identify and remediate similar patterns elsewhere in the application.

13. Limitations

- This assessment was performed by an entry-level tester as a learning exercise and does not represent a comprehensive, exhaustive penetration test.
- Testing focused on a limited set of manual checks rather than full application coverage.
- Only one vulnerability class was validated in depth; other potential issues may exist but were not tested.

14. Conclusion

This assessment successfully identified and validated a critical SQL Injection vulnerability leading to authentication bypass in OWASP Juice Shop's login functionality. The finding was documented using an evidence-based approach consistent with professional VAPT reporting practices, demonstrating foundational skills in reconnaissance, enumeration, request analysis, and vulnerability validation.

15. Disclaimer

This assessment was conducted exclusively against a locally hosted, intentionally vulnerable training application (OWASP Juice Shop) within a private, isolated lab environment. No unauthorized access or testing was performed against any third-party, live, or production system. This report was prepared solely for educational and portfolio purposes.